06_route 스크립트는 Synopsys **IC Compiler II (ICC2)** 환경에서 칩 내부의 모든 신호선(Signal Net)을 물리적인 메탈 배선으로 완벽하게 연결하는 **Routing (**라우팅**,** 배선**)** 단계의 전체 Tcl 스크립트입니다.

CTS 단계 완료 후 고정된 클럭 트리 구조 위에서, 수만 개의 셀 간의 신호 배선을 글로벌 라우팅, 트랙 할당, 디테일 라우팅 거쳐 DRC/LVS 검증까지 완료하는 최종 핵심 공정입니다.

---

1. 초기 단계 선언 및 라이브러리 로드

tcl

```
#made by kjinsoo
# module load
#module load icc2/2021.06-SP5
# open icc2_shell
#icc2_shell -gui | tee -i ./07_route.log

#set current step & before step
set step route
set before_step psyn
#source design_setup.tcl
source -e ./icc2_common_env/design_setup.tcl

#source timer.rpt script - report runtime for placement
source ./icc2_common_env/sec_runtime.tcl

sec_StartTimer copy_lib

#set_host_options -max_cores 16

#open lib
open_lib ../05_cts/nlib/ORCA_${before_step}.nlib
#copy lib
copy_lib -from ../05_cts/nlib/ORCA_${before_step}.nlib -to
./nlib/ORCA_${step}.nlib
#close & open current lib
close_lib -all
open_lib ./nlib/ORCA_${step}.nlib
#open block
open_block ./nlib/ORCA_${step}.nlib:ORCA.design
#link_block
link_block -rebind

sec_ReportResourceUtil copy_lib
```

코드를 사용할 때는 주의가 필요합니다.

- 해설: 라우팅 단계 실행 로그(`07_route.log`) 및 단계 변수를 선언합니다. 이전 물리적 최적화 단계(`psyn`)의 결과물 라이브러리를 카피하여 `route` 독립 라이브러리로 구성합니다. `link_block -rebind`를 통해 논리 망과 물리적 셀 인스턴스를 재동기화합니다.

---

2. MCMM 환경 구성 및 타이밍 제약 조건(SDC) 로드

tcl

```
## Setup
####################################
#remove scenarios modes corners before setting
remove_scenarios -all
remove_modes -all
remove_corners -all

# Read TLU+ files
read_parasitic_tech -name {minTLU} -tlup $data(tech_tlupbc)
read_parasitic_tech -name {maxTLU} -tlup $data(tech_tlupwc)

# source mcmm(multi corners multi modes).tcl
source -e ./icc2_common_env/mcmm.tcl

#read_sdc(synopsys design constraints)
#scan_shift mode
current_mode scan_shift
read_sdc $data(shift_sdc) ;# ${libdir}/SDC/ORCA_shift.sdc

#func mode
current_mode func
read_sdc $data(func_sdc) ;# ${libdir}/SDC/ORCA_func.sdc

#scan_capture mode
current_mode scan_capture
read_sdc $data(capture_sdc) ;# ${libdir}/SDC/ORCA_capture.sdc
```

코드를 사용할 때는 주의가 필요합니다.

- 해설: 라우팅 단계에서의 정밀한 배선 정전용량(RC) 계산을 위해 기존 타이밍 코너 설정을 리셋하고, 파운드리의 TLU+ 룩업 테이블 파일(`minTLU`, `maxTLU`)과 MCMM 시나리오를 다시 주입합니다. 3가지 주요 모드(`scan_shift`, `func`, `scan_capture`)의 SDC 타이밍 제약 조건을 완벽히 로드합니다.

---

3. 안테나 효과 규칙 적용 및 홀드 방어용 셀 지정

tcl

```
#set anttena rule
source -echo ./icc2_common_env/saed32nm_ant_1p9m.tcl

#set include hold Timing optimization cell
set_lib_cell_purpose -exclude hold [get_lib_cells]
set_lib_cell_purpose -include hold [get_lib_cells "*/NBUFFX2_HVT
saed32hvt/DELLN1X2_HVT"]
```

코드를 사용할 때는 주의가 필요합니다.

- **`saed32nm_ant_1p9m.tcl`:** 제조 공정 중 메탈 배선이 길어지면 정전기가 모여 게이트 산화막을 파괴하는 안테나 효과**(Antenna Effect)**가 발생합니다. 이를 예방하기 위해 다이오드나 버퍼를 강제로 심도록 규정하는 공정 안테나 룰을 로드하는 아주 중요한 단계입니다.
- **`set_lib_cell_purpose ...`:** 라우팅 마무리 단계에서 발생할 미세 홀드(Hold) 타이밍 위반을 고치기 위해 지정된 특정 고임계 전압(HVT) 버퍼 및 지연 셀(`NBUFFX2_HVT`, `DELLN1X2_HVT`)만 사용하도록 허용 범위를 제한합니다.

---

4. 실제 클럭 지연 반영 (Propagated Clock 적용)

tcl

```
#remove clock_latency & ideal_network
set mode "func scan_shift scan_capture"

foreach MODE $mode {
current_mode $MODE
echo "current mode is $MODE"
remove_clock_latency [get_clocks *]
remove_clock_latency [get_flat_pins -filter "is_clock_pin == true"]
remove_ideal_network [remove_from_collection [all_fanout -flat
-clock_tree] [get_ports [all_output]]]
set_propagated_clock [all_clocks]
}

compute_clock_latency
```

코드를 사용할 때는 주의가 필요합니다.

- **`remove_clock_latency / remove_ideal_network`:** CTS가 완료되었으므로 기존의 가상 클럭 속성(Ideal Network)과 추정치 레이턴시 값을 전국의 모든 클럭 핀 및 출력 포트 제외 트리에서 완벽하게 제거합니다.
- **`set_propagated_clock [all_clocks]`:** 이제부터는 가상이 아닌, 실제 **CTS** 단계를 통해 심어진 버퍼와 메탈 선의 물리적 저항/정전용량을 추적하여 클럭 지연을 셈하라고 명령하는 핵심 구문입니다.
- **`compute_clock_latency`:** 칩 전체의 실제 클럭 트리 네트워크 배선 상태를 기반으로 정밀한 물리적 클럭 레이턴시 값을 최종 계산하여 동기화합니다.

---

5. 클럭 경로 최대 트랜지션(Slew) 제약 설정

tcl

```
#set max_transition for all Scenarios
current_mode func
set_max_transition 0.5 -clock_path [get_clocks] -corners [all_corners]

current_mode scan_capture
set_max_transition 0.5 -clock_path [get_clocks] -corners [all_corners]

current_mode scan_shift
set_max_transition 0.5 -clock_path [get_clocks] -corners [all_corners]
```

코드를 사용할 때는 주의가 필요합니다.

- 해설: 클럭 신호의 파형이 뭉개지면 노이즈와 지터(Jitter)에 취약해집니다. 이를 막기 위해 세 가지 모든 동작 모드와 모든 코너에서 클럭 배선 경로의 신호 상승/하강 시간(Transition/Slew)이 최대 **0.5ns(500ps)**를 초과하지 않도록 강력한 제약을 겁니다.

---

6. 라우팅 최적화 옵션(Application Options) 튜닝

tcl

```
################################
#set_app_options before routing
################################
set_app_options -list {opt.tie_cell.add_to_highest_hierarchy {false}}
set_attribute -objects [get_blocks -all -lib_cells
saed32hvt:TIEH_HVT.frame] -name dont_touch -value false
set_attribute -objects [get_blocks -all -lib_cells
saed32hvt:TIEL_HVT.frame] -name dont_touch -value false

set_app_options -list {route.global.timing_driven {true}}
```

```
set_app_options -list {route.global.crosstalk_driven {true}}
set_app_options -list {route.track.timing_driven {true}}
set_app_options -list {route.track.crosstalk_driven {true}}
set_app_options -list {route.detail.timing_driven {true}}
set_app_options -list {route.detail.force_max_number_iterations
{false}}

set_app_options -list {route.detail.generate_extra_off_grid_pin_tracks
{false}}
set_app_options -list
{route.detail.generate_off_grid_feed_through_tracks {high}}
```

코드를 사용할 때는 주의가 필요합니다.

- **`dont_touch ... false`**: 전압 고정용 TIE 셀(`TIEH`, `TIEL`)의 보호 지정을 해제하여, 배선 최적화 과정에서 툴이 이를 자유롭게 삽입하고 조율할 수 있게 만듭니다.
- **`timing_driven / crosstalk_driven {true}`**: 글로벌 라우팅, 트랙 할당, 디테일 라우팅의 모든 배선 단계에서 타이밍 지연 최소화뿐만 아니라 인접 배선 간의 간섭 노이즈인 크로스토크**(Crosstalk)** 현상까지 동시에 고려하여 똑똑하게 선을 깔도록 설정합니다.
- **`generate_off_grid_feed_through_tracks {high}`**: 매크로 등 차단 영역(Blockage) 사이에 신호선이 지나갈 때, 격자(Grid)를 벗어나더라도 우회로 배선 트랙을 만들어낼 수 있도록 엔진의 연산 강도(`high`)를 높여 배선 성공률을 극대화합니다.

---

7. 자동 전체 라우팅 1차 실행 (`route_auto`)

tcl

```
############################
#route_auto (detail route)
############################
#detail_route
sec_StartTimer route

route_auto ;# The route_auto command performs all routing stages in a
single step (global routing, track assignment, detail routing)

sec_ReportResourceUtil route
```

코드를 사용할 때는 주의가 필요합니다.

- **`route_auto`:** 라우팅 단계의 메인 총괄 명령어입니다. 가상의 통로를 개척하는 **1) Global Routing**, 실제 메탈 트랙 위에 선을 안착시키는 **2) Track Assignment**, 공정 규칙(DRC)을 준수하며 정밀 결선하는 **3) Detail Routing**까지의 전 과정을 원스톱으로 자동 실행합니다.

---

8. 배선 개선(Routability) 최적화 및 2차 라우팅 실행

tcl

```
#set app option for second route_auto
set_app_options -list {place.legalize.stream_place {true}}

#optimize_routability
optimize_routability

#second route_auto
sec_StartTimer route2

route_auto

sec_ReportResourceUtil route2

set_app_options -list {place.legalize.stream_place {false}}
```

코드를 사용할 때는 주의가 필요합니다.

- **`stream_place {true}` / `optimize_routability`:** 1차 배선 완료 후 메탈선이 너무 조밀하게 꼬여 DRC 위반이 대량 발생한 구역을 해소하기 위해, 스탠다드 셀들을 미세하게 밀어내거나 뒤집어(Flipping) 배선 공간 마진을 확보하는 튜닝 명령어입니다.
- 확보된 공간을 바탕으로 **`route_auto`**를 **2**차 재실행하여 남은 미세 배선 오픈/쇼트 결함들을 말끔히 지워 나갑니다.

---

9. 미세 수정(Search & Repair) 및 타이밍 확정

tcl

```
#Search & Repair
route_detail -incremental true -initial_drc_from_input true
compute_clock_latency
update_timing
```

코드를 사용할 때는 주의가 필요합니다.

- **`route_detail -incremental true`:** 이른바 **Search & Repair(**탐색 및 수리**)** 공정입니다. 전체를 다시 까는 것이 아니라, 국소적으로 남은 미세 DRC 위반 지점만 싹 찾아내어 지우고 다시 연결하는 정밀 보정 작업입니다.
- 미세 수정이 끝났으므로 실제 배선 저항이 반영된 최종 클럭 레이턴시(`compute_clock_latency`)와 전체 칩 타이밍 그래프(`update_timing`)를 확정 업데이트합니다.

---

10. 전원선 마무리 결선 (PG Connection)

tcl

```
##connect_pg_net (CORNER_CELL & FILLER_CELL)
source -e ./user_script/pg_connection.tcl
```

코드를 사용할 때는 주의가 필요합니다.

- 해설**:** 라우팅 중간 혹은 웰(Well) 타이이를 채우기 위해 빈 공간에 배치된 코너 셀(Corner Cell) 및 피용 필러 셀(Filler Cell)의 내부 전원 핀을 메인 VDD/VSS 전원 패브릭에 확실하게 묶어주는 수동 PG 결선 스크립트를 수행합니다.

---

11. 배선 상태(Route) 및 물리 레이아웃(LVS) 최종 검증

tcl

```
#check_route after route_auto
check_route

#check_lvs (short, open, floating)
check_lvs
```

코드를 사용할 때는 주의가 필요합니다.

- **`check_route`:** 미연결 배선(Open), 겹친 배선(Short)의 유무와 레이어별 배선 혼잡도 통계를 최종 리포트합니다.
- **`check_lvs`:** Netlist 회로도 구조와 실제 메탈 레이아웃 구조가 완벽히 일치하는지 검증하는 **LVS(Layout Vs Schematic)**를 수행하여 선이 허공에 뜬 곳(Floating)이나 단선, 쇼트 유반을 전수 조사합니다. (테이프 아웃 전 필수 검증)

---

12. 최종 성적표 출력 및 데이터베이스 마감 저장

tcl

```
#report_timing & QoR
report_timing -significant_digits 5
report_qor -summary -significant_digits 5

############################################################
#report Timing & constraints for route_auto
############################################################
#update timing
update_timing -full

sec_StartTimer report
#write "reports files" of psyn at ${rundir}/reports/psyn/* directory
source ./user_script/route_rpt.tcl
sec_ReportResourceUtil report

#Save & Exit
save_lib -all
exit
```

코드를 사용할 때는 주의가 필요합니다.

- 모든 배선과 레이아웃 검증이 성공적으로 마무리되었으므로, 소수점 5자리 단위의 초정밀 타이밍 리포트(`report_timing`) 및 칩 종합 요약본(`report_qor`)을 출력합니다.
- `route_rpt.tcl` 스크립트를 통해 지정 경로에 최종 배선 성적표 파일들을 일괄 저장한 뒤, 가공된 모든 라이브러리를 안전하게 백업 디스크에 쓰고(`save_lib -all`) ICC2 쉘을 최종 종료합니다. 이 단계가 성공하면 칩 제조 공장에 레이아웃(GDSII/OASIS)을 넘겨주기 전 마지막 공정인 **Sign-off (**최종 검증**)** 단계로 진입하게 됩니다.

🚀 디지털 백엔드(Back-End) 스크립트 리뷰 요약

보내주신 4개의 스크립트를 통해 **Power Plan ➔ Placement ➔ CTS ➔ Routing**으로 이어지는 실제 반도체 설계 자동화(EDA) 핵심 메인 플로우가 완벽하게 빌드되어 있음을 확인할 수 있었습니다.